

Serial

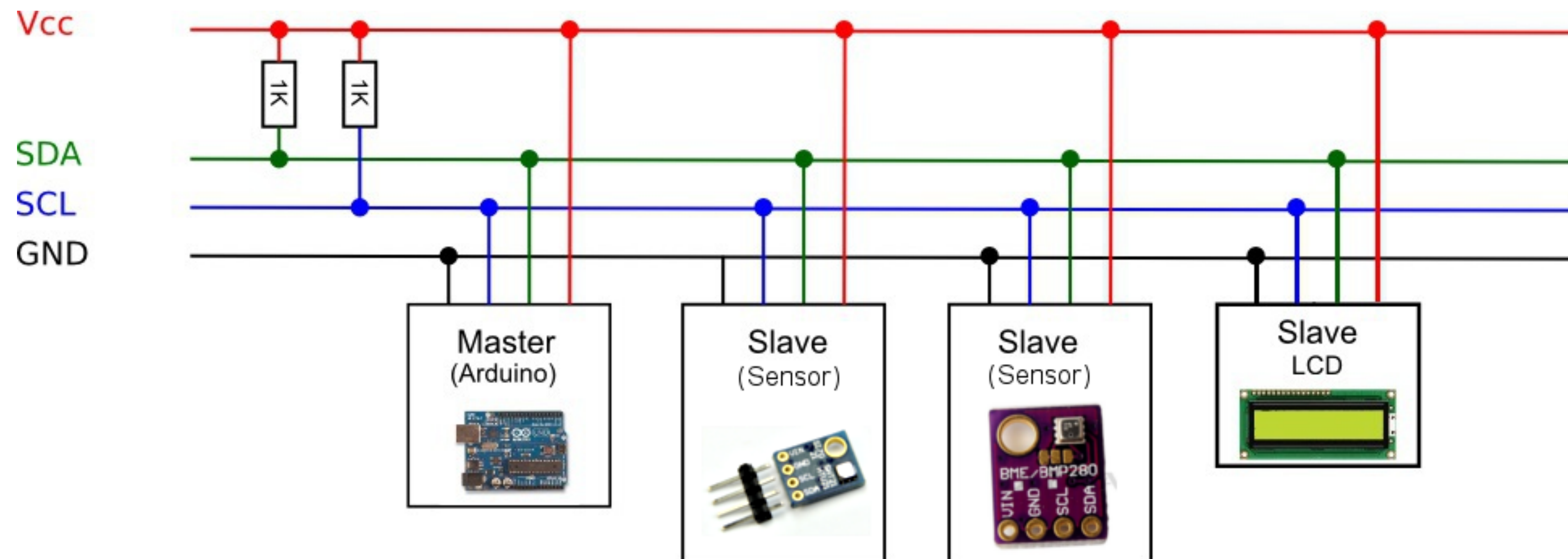
Transfert de données digitale

- Les communications synchrones sont plus rapides
- Plus robustes
- Il existe de multiples protocoles **synchrones** :
 - SPI
 - I2C
 - PCI Express
 - Serial USB
 - ...

Serial

I2C

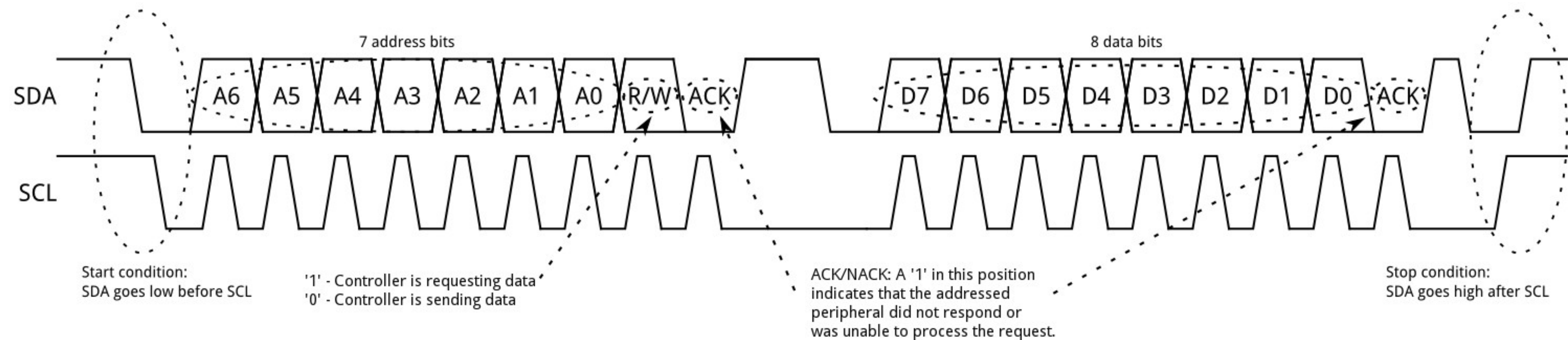
- Inter-Integrated Circuit
- But : Communiquer entre différents composants d'un même circuit
- C'est un **bus** de communication : Il permet de communiquer avec plusieurs devices
- Un Device est maître et ainsi s'occupe d'envoyer ou recevoir les données
- Pour les courtes distances (1m max)



Serial

I2C

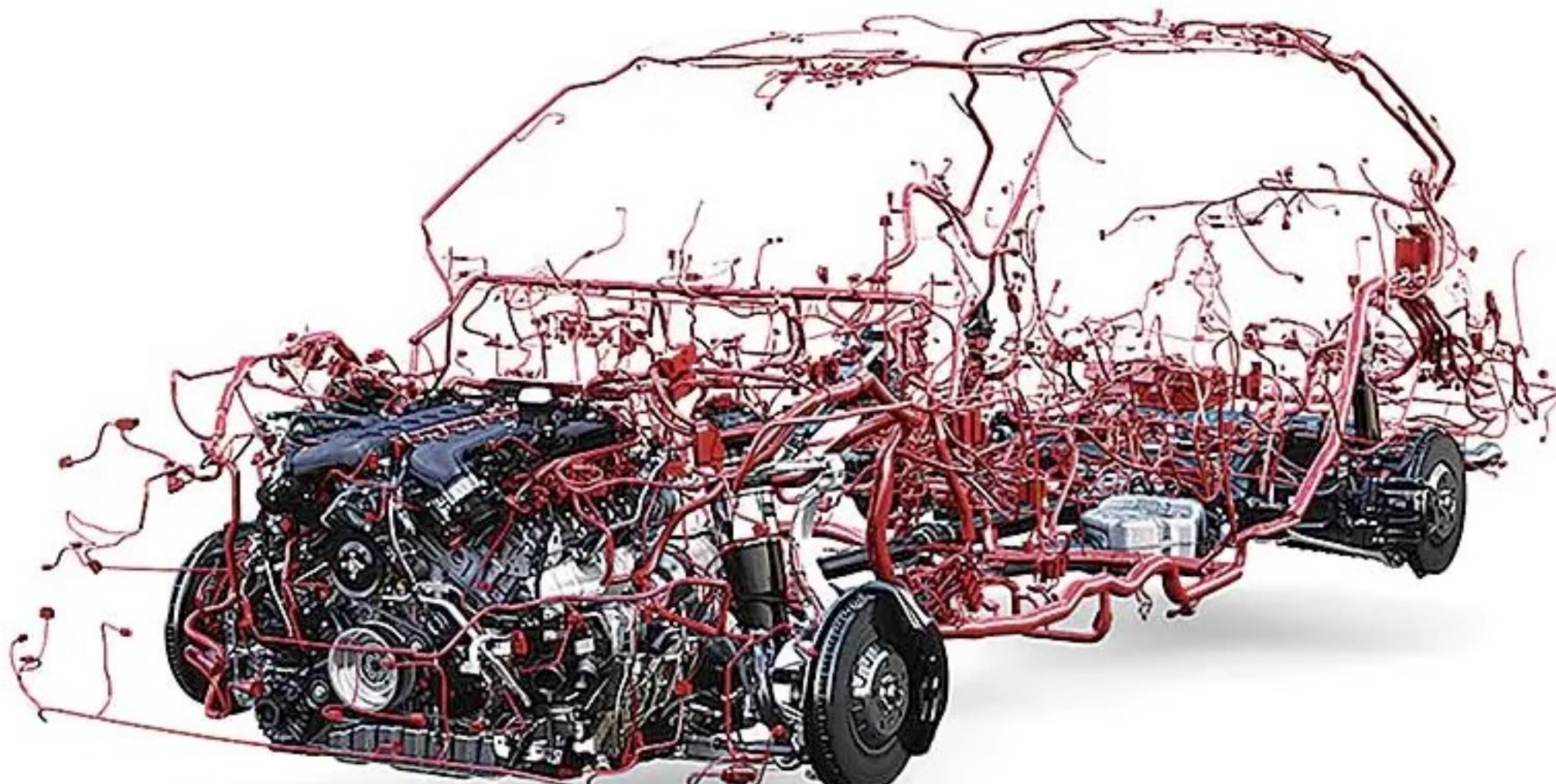
- On adresse le composant avec lequel on veut communiquer, et si on veut écrire ou lire ses données (R/W bit)
- On attend un ACK (Acknowledge) pour vérifier que le composant existe
- On envoie la donnée
- On vérifie que la donnée a été reçue avec un ACK



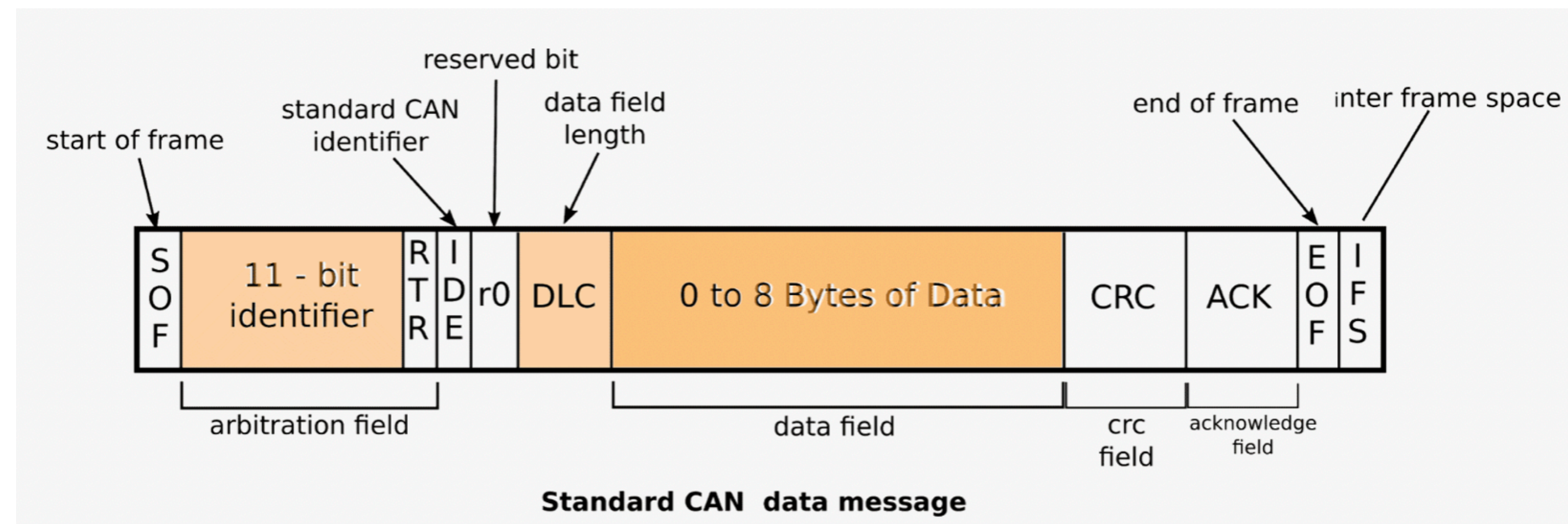
- Un adresse peut être en 8-bit ou 16-bit (dans ce cas on envoie une grande trame d'adresse qui commence de façon spécifique)

Serial CanBus

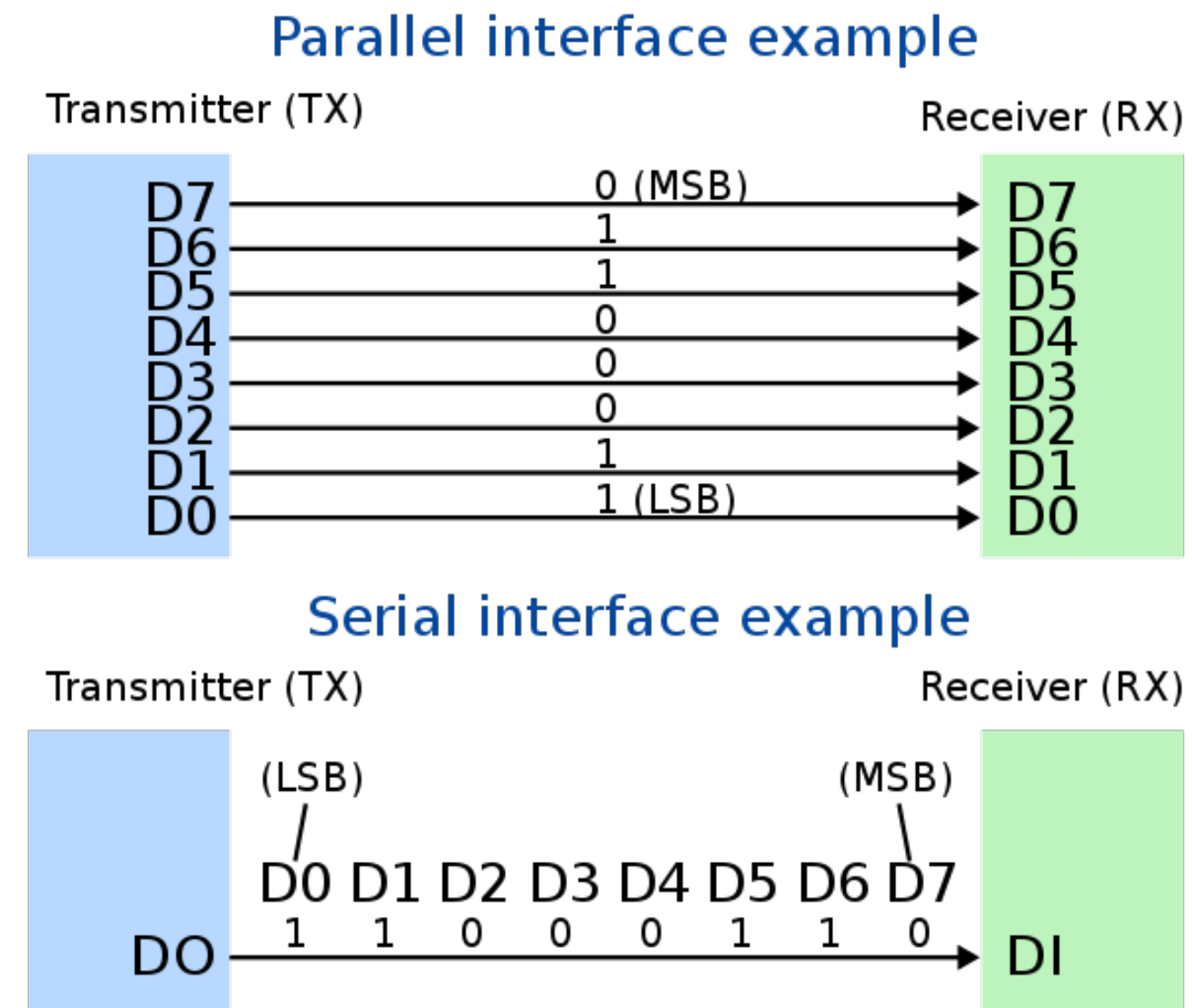
- Le CANBUS est un protocole utilisé principalement dans les voitures
- Démocratisé dans les systèmes lourds et complexes
- Il est **asynchrone (contrairement à l'I2C)**
- Le CANBUS a besoin de plusieurs composants pour fonctionner
 - Les signaux peuvent avoir des tensions différentes
 - Les signaux sont basés sur la différences de tension entre les câbles
- I2C est fait pour les courtes distances, contrairement au CAN bus (1km)
- Le principe est de broadcaster dans le bus un message, qui peut donc être lu par n'importe qui



Transparency Market Research



Parallel Communication



- Serial vs Parallel :
 - Permet d'envoyer plusieurs bits d'un coup
 - N'est pas forcément mieux, car interfaçage est plus complexe, plus coûteux
- Cas d'usage :
 - Fibre optique
 - Anciens protocols

- *from microbit import*
- *i2c.write(address, data) : envoie de la trame sur le bus I2C*
- *i2c.read(address, data) : lecture sur le bus I2C*

- ***Ici on travaille avec un I2C Register Description (IRD) :***
 - ***Chaque fonctionnalité est accessible en envoyant une adresse dans la data***
 - ***L’adresse du registre : 0x10***
 - ***La data à envoyer : [Adresse, (data)]. Pour une lecture : write [Adresse], read***

Élément	Rôle	Grandeur	Adresse	Octet
Moteur gauche	Contrôle direction	Direction	0	0 arrêt 1 avant
Moteur gauche	Contrôle vitesse	Vitesse	1	0-255
Moteur droit	Contrôle direction	Direction	2	0 arrêt 1 avant
Moteur droit	Contrôle vitesse	Vitesse	3	0-255
Codeur roue gauche	Nombre de tour de roue	Tours	4	
Codeur roue droite	Nombre de tour de roue	Tours	6	
Moteur gauche	Compensation de la vitesse du moteur pour corriger les défauts dus à la route, à la roue ou à la vitesse	Compensation vitesse	8	0-255 (0-1tr/min)
Moteur droit	Compensation de la vitesse du moteur pour corriger les défauts dus à la route, à la roue ou à la vitesse	Compensation vitesse	9	0-255 (0-1tr/min)
Commutateur PID	Lorsque le PID est activé, le moteur a une vitesse plus précise et le couple est plus élevé	PID	A	0 désactivé 1 activé
Led RGB gauche	Contrôle LED RGB	Couleur	B	0-7 0 Éteint, 1 rouge, 2 vert
Led RGB droite	Contrôle LED RGB	Couleur	C	0-7
Servomoteur connecteur S1	Contrôle du servomoteur	Angle	14	0-180 (170 maxi recommandé)
Servomoteur connecteur S2	Contrôle du servomoteur	Angle	15	0-180 (170 maxi recommandé)
Servomoteur connecteur S3	Contrôle du servomoteur	Angle	16	0-180 (170 maxi recommandé)

Capteurs de suivi de ligne, état	Indique la présence d 'une ligne noire L'état des 6 capteurs est stocké dans un octet	État	1D	ex : b'0000 1100' 1 pour du noir, 0 pour du blanc
Capteurs de suivi de ligne, niveau de gris	L3 (1) Indique le niveau de gris du capteur sur 2 octets	Niveau de gris	0x1E 0X20	Plage de retour : 0-4095
	L2 (2)		0x21 0x22	
	L1 (3)		0x23 0x24	
	R1 (4)		0x25 0x26	
	R2 (5)		0x27 0x28	
	R3 (6)		0x29 0x2A	
Version produit			0x32 0x33	